

TOPS TECH MODUAL-1 Assessment

Linux Server Security and Automation

THEME: STARTUP WEB SERVER DEPLOYMENT

SECTION: A Scenario-Based Concept Application

1. **Explain the operational factors that justify selecting a specific Linux distribution for a startup's internal tool hosting environment.**

→ When choosing a Linux distribution for a startup's internal tool hosting environment, the following operational factors are considered:

1. Cost / Licensing:

→ Ubuntu LTS and Debian are completely free. RHEL offers enterprise support but requires paid licensing. Startups prefer cost-free distros to minimize overhead.

→ Community and Support:

→ Ubuntu has the largest community. CentOS/AlmaLinux mirrors RHEL behaviour and is preferred in enterprise-like environments without licensing cost.

2. Package Management:

→ Ubuntu uses APT (apt-get), RHEL/CentOS uses YUM/DNF. APT is generally easier for developers; DNF is more structured for production servers.

3. Security Updates:

→ Ubuntu LTS versions receive 5 years of security patches. RHEL provides 10 years. Long-term support is critical for production servers.

4. Compatibility:

→ Ubuntu is widely compatible with Docker, Kubernetes, and most web hosting tools. CentOS/RHEL is standard in enterprise and government environments.

TOPS TECH MODUAL-1 Assessment

5. Hardware and Cloud Support:

→ Ubuntu and Debian have excellent support on AWS, GCP, and Azure. Most cloud providers provide pre-configured images for Ubuntu.

6. Team Expertise:

→ If the development team already knows a specific distro, deployment and troubleshooting will be faster and safer.

2. Explain the implementation of SSH Key-based Authentication to satisfy a security requirement where password logins must be disabled for developer access.

SSH Key-based Authentication uses a cryptographic key pair (public + private key) instead of passwords. Here is the full implementation:

Step 1 – Generate Key Pair on Developer Machine:

```
ssh-keygen -t rsa -b 4096 -C "developer@startup.com"
```

This creates ~/.ssh/id_rsa (private key) and ~/.ssh/id_rsa.pub (public key).

Step 2 – Copy Public Key to Server:

```
ssh-copy-id dev-ops@<server-ip>
```

OR manually append public key to server's ~/.ssh/authorized keys.

Step 3 – Edit SSH Configuration on Server:

```
Sudo nano /etc/ssh/sshd_config
```

Make the following changes:

```
PubkeyAuthentication yes
```

```
Password Authentication no
```

```
PermitRootLogin no
```

Step 4 – Restart SSH Service:

```
sudo systemctl restart sshd
```

TOPS TECH MODUAL-1 Assessment

Step 5 – Test Login:

```
ssh -i ~/.ssh/id_rsa dev-ops@<server-ip>
```

Security Benefit: Without the private key file, no one can log in — even if they know the username and password. Brute-force attacks become completely ineffective.

3. Explain the configuration of Linux Permissions and Ownership for the /var/www directory to support a multi-user environment.

In a multi-user web environment, the /var/www directory must be accessible to web developers while remaining secure. Here is the proper configuration:

Step 1 – Create a Shared Group:

```
sudo groupadd webadmins  
sudo usermod -aG webadmins dev-ops  
sudo usermod -aG webadmins www-data
```

Step 2 – Set Ownership:

```
sudo chown -R root:webadmins /var/www
```

root owns the directory, the webadmins group has group-level access.

Step 3 – Set Permissions:

```
sudo chmod -R 775 /var/www
```

7 = owner (root) has full access, 7 = group (webadmins) has full access, 5 = others have read+execute only.

Step 4 – Set SetGID Bit (Group Inheritance):

```
sudo chmod g+s /var/www/html
```

This ensures that new files created in the directory automatically inherit the webadmins group.

TOPS TECH MODUAL-1 Assessment

Step 5 – Verify Permissions::

```
ls -la /var/www
```

This setup allows multiple developers to create and edit web files without interfering with each other and without needing root access.

4. **Explain how Logical Volume Management (LVM) facilitates dynamic storage expansion to prevent service downtime when disk space is depleted.**

LVM (Logical Volume Management) is a storage management layer in Linux that allows flexible disk space management without unmounting partitions or stopping services.

LVM Architecture:

- PV (Physical Volume): Raw disk or partition (e.g., /dev/sdb)
- VG (Volume Group): Pool of storage from one or more PVs
- LV (Logical Volume): Virtual partition carved from VG

How to Expand Storage Dynamically (Without Downtime):

Step 1 – Add new disk/partition:

```
sudo pvcreate /dev/sdc
```

Step 2 – Extend the Volume Group:

```
sudo vgextend my_vg /dev/sdc
```

Step 3 – Extend the Logical Volume:

```
sudo lvextend -L +20G /dev/my_vg/my_lv
```

Step 4 – Resize the filesystem (no reboot needed):

```
sudo resize2fs /dev/my_vg/my_lv
```

Key Benefit:

→ The entire process can be done while the server is running and the web service is live. There is zero downtime, which is critical for production environments.

TOPS TECH MODUAL-1 Assessment

LVM also supports snapshots for backup, which allows administrators to create point-in-time copies of volumes without stopping services.

5. **Explain the troubleshooting process for diagnosing and resolving service blocks caused by SELinux security policies when a website becomes inaccessible.**

SELinux (Security-Enhanced Linux) enforces mandatory access control policies. It can sometimes block Apache from serving files even when standard Linux permissions are correct.

Step 1 – Check SELinux Status:

```
sestatus
```

```
getenforce
```

Output will be: Enforcing, Permissive, or Disabled.

Step 2 – Check SELinux Audit Logs:

```
sudo cat /var/log/audit/audit.log | grep denied
```

```
sudo ausearch -m AVC -ts recent
```

Look for 'denied' messages related to httpd (Apache).

Step 3 – Use audit2why to Explain the Block::

```
sudo ausearch -m AVC | audit2why
```

Step 4 – Fix SELinux Context for Web Files:

```
sudo chcon -R -t httpd_sys_content_t /var/www/html
```

```
sudo restorecon -Rv /var/www/html
```

Step 5 – Allow Apache to make network connections (if needed):

```
sudo setsebool -P httpd_can_network_connect 1
```

Step 6 – Temporarily set to Permissive mode (for testing only):

```
sudo setenforce 0
```

Warning: Never leave SELinux in permissive mode on production servers. Always fix the policy properly.

TOPS TECH MODUAL-1 Assessment

Step 7 – Generate Custom Policy (advanced):

```
sudo ausearch -m AVC | audit2allow -M myapache  
sudo semodule -i myapache.pp
```

6. Explain the safe configuration of Cron Jobs to ensure critical daily backup tasks execute automatically at a scheduled time.

Cron is a Linux time-based job scheduler. To safely configure a daily backup cron job, follow these steps:

Step 1 – Create the Backup Script:

```
sudo nano /usr/local/bin/backup_www.sh
```

Script content:

```
#!/bin/bash  
DATE=$(date +%Y%m%d_%H%M%S)  
tar -czf /backup/www_backup_$DATE.tar.gz /var/www  
echo "Backup completed: $DATE" >> /var/log/backup.log
```

Step 2 – Make Script Executable:

```
sudo chmod +x /usr/local/bin/backup_www.sh
```

Step 3 – Open Crontab for Root::

```
sudo crontab -e
```

Step 4 – Add Cron Entry (runs at 02:00 AM daily):

```
0 2 * * * /usr/local/bin/backup_www.sh
```

TOPS TECH MODUAL-1 Assessment

Step 5 – Verify Crontab Entry:

```
sudo crontab -l
```

Step 6 – Check Cron Logs:

```
grep CRON /var/log/syslog
```

Safety Best Practices:

- Always redirect script output to a log file for debugging
- Test the script manually before adding to cron
- Store backups on a separate disk or remote location
- Use absolute paths in cron scripts (cron has a limited PATH)
- Set MAILTO="" to suppress email output if not needed

SECTION B: Practical Application

Tools Required: Linux (Ubuntu/CentOS/RHEL), Terminal/CLI, Apache Web Server, UFW/Firewalld

B1. System Identity – Set Hostname to startup-web-01

Command to set hostname:

```
sudo hostnamectl set-hostname startup-web-01
```

Verify the change:

```
hostnamectl
```

```
hostname
```

Also, update /etc/hosts to reflect the new name:

```
sudo nano /etc/hosts
```

```
127.0.0.1 startup-web-01
```

TOPS TECH MODUAL-1 Assessment

B2. Network Setup – Configure Static IP Address

For Ubuntu (using Netplan):

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Add the following configuration:

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses: [192.168.1.100/24]
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
```

Apply the configuration:

```
sudo netplan apply
```

Verify:

```
ip addr show ens33
```

For CentOS/RHEL (using nmcli):

```
sudo nmcli con mod ens33 ipv4.addresses 192.168.1.100/24
```

```
sudo nmcli con mod ens33 ipv4.gateway 192.168.1.1
```

```
sudo nmcli con mod ens33 ipv4.method manual
```

```
sudo systemctl restart NetworkManager
```

B3. Secure Access – Enable SSH Service on Boot

```
sudo systemctl enable ssh      # Ubuntu
```

```
sudo systemctl enable sshd     # CentOS/RHEL
```

```
sudo systemctl start sshd
```


TOPS TECH MODUAL-1 Assessment

```
sudo systemctl status sshd
```

Verify SSH is listening:

```
ss -tlnp | grep 22
```

B4. Access Control – Create Group webadmins and User dev-ops

Create group and user:

```
sudo groupadd webadmins
sudo useradd -m -s /bin/bash dev-ops
sudo usermod -aG webadmins dev-ops
```

Set password for dev-ops:

```
sudo passwd dev-ops
```

Restrict sudo privileges (edit sudoers):

```
sudo visudo
```

Add this line to allow only specific commands:

```
dev-ops ALL=(ALL) /usr/bin/systemctl restart apache2,
/usr/bin/systemctl status apache2
```

Verify:

```
groups dev-ops
sudo -l -U dev-ops
```

B5. Web Hosting – Install Apache and Deploy index.html

Install Apache:

```
sudo apt update && sudo apt install apache2 -y # Ubuntu
sudo yum install httpd -y # CentOS/RHEL
```

Start and enable Apache:

```
sudo systemctl start apache2
```

TOPS TECH MODUAL-1 Assessment

```
sudo systemctl enable apache2
```

Deploy custom index.html:

```
sudo nano /var/www/html/index.html
```

Add content:

```
<html><body><h1>Welcome to Startup Web  
Server</h1></body></html>
```

Set correct permissions:

```
sudo chown www-data:www-data /var/www/html/index.html  
sudo chmod 644 /var/www/html/index.html
```

Verify in browser: `http://<server-ip>`

B6. Hardening – Configure Firewall (Allow Only Ports 22 and 80)

Using UFW (Ubuntu):

```
sudo ufw default deny incoming  
sudo ufw default allow outgoing  
sudo ufw allow 22/tcp  
sudo ufw allow 80/tcp  
sudo ufw enable  
sudo ufw status verbose
```

Using FirewallD (CentOS/RHEL):

```
sudo systemctl start firewalld  
sudo systemctl enable firewalld  
sudo firewall-cmd --permanent --add-service=ssh  
sudo firewall-cmd --permanent --add-service=http  
sudo firewall-cmd --permanent --remove-service=dhcpv6-client  
sudo firewall-cmd --reload  
sudo firewall-cmd --list-all
```

B7. Automation – Cron Job to Back Up /var/www at 02:00 Daily

TOPS TECH MODUAL-1 Assessment

Create backup directory:

```
sudo mkdir -p /backup
```

Create backup script:

```
sudo nano /usr/local/bin/backup_www.sh
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
tar -czf /backup/www_backup_${DATE}.tar.gz /var/www 2>/dev/null
echo "Backup done at $DATE" >> /var/log/www_backup.log
```

Make it executable:

```
sudo chmod +x /usr/local/bin/backup_www.sh
```

Add to crontab:

```
sudo crontab -e
0 2 * * * /usr/local/bin/backup_www.sh
```

Verify:

```
sudo crontab -l
ls /backup/
```

SECTION C: Integrated Case Study

Scenario: *"CloudScale" performed a kernel update. After reboot: 502 Gateway Error on web app, SSH login takes 60+ seconds. The server is live, but the business is losing revenue.*

→ **Diagnosis – Which log files and commands to identify why Apache failed to start?**

Primary Command:

```
sudo systemctl status apache2
sudo systemctl status httpd
```

TOPS TECH MODUAL-1 Assessment

Detailed Logs:

```
sudo journalctl -u apache2 --no-pager -n 50  
sudo journalctl -xe | grep apache
```

Apache Error Logs:

```
sudo cat /var/log/apache2/error.log  
sudo tail -100 /var/log/apache2/error.log
```

System Boot Logs:

```
sudo dmesg | grep -i error  
sudo journalctl -b -p err
```

Port Conflict Check:

```
sudo ss -tlnp | grep:80  
sudo lsof -i :80
```

Config Syntax Check::

```
sudo apachectl configtest  
sudo apache2ctl -t
```

Common Causes After Kernel Update:

- Port 80 already in use by another service
- SELinux context changed after kernel update
- Missing kernel module or library dependency
- Apache config file corrupted or incompatible

→ **Connectivity – Why is SSH login experiencing high latency (60+ seconds)? Suggest a fix.**

High SSH login latency is almost always caused by one of these issues:

Cause 1 – Reverse DNS Lookup (Most Common):

TOPS TECH MODUAL-1 Assessment

→ When SSH connects, it tries to do a reverse DNS lookup of the client IP. If DNS is slow or unreachable, this causes a 60-second timeout.

Fix – Disable DNS Lookup in SSH Config:

```
sudo nano /etc/ssh/sshd_config  
UseDNS no  
sudo systemctl restart sshd
```

Cause 2 – GSSAPI Authentication Timeout:

→ SSH tries GSSAPI (Kerberos) auth before falling back to key auth. This causes a delay.

Fix – Disable GSSAPI::

```
GSSAPIAuthentication no  
sudo systemctl restart sshd
```

Cause 3 – Network Manager DNS Issues After Kernel Update:

```
sudo systemctl restart systemd-resolved  
cat /etc/resolv.conf
```

Cause 4 – PAM Authentication Delay:

```
sudo nano /etc/pam.d/sshd
```

Comment out lines related to pam_motd or pam_nologin if present after kernel update.

Verification::

```
ssh -v user@server # Verbose mode shows where the delay occurs
```

3. Recovery – Step-by-step CLI process to restart failed services and verify status.

TOPS TECH MODUAL-1 Assessment

Step 1 – Check what's running:

```
sudo systemctl list-units --state=failed
```

Step 2 – Restart Apache:

```
sudo systemctl restart apache2    # Ubuntu  
sudo systemctl restart httpd      # CentOS/RHEL
```

Step 3 – Verify Apache is running:

```
sudo systemctl status apache2  
curl -I http://localhost
```

Step 4 – Restart SSH if needed:

```
sudo systemctl restart sshd  
sudo systemctl status sshd
```

Step 5 – Check if the web app responds:

```
curl http://localhost  
wget -O- http://localhost
```

Step 6 – Verify listening ports:

```
sudo ss -tlnp | grep -E '80|443|22'
```

Step 7 – Check the firewall is not blocking:

```
sudo ufw status    # Ubuntu  
sudo firewall-cmd --list-all # CentOS
```

Step 8—Enable services to auto-start:

```
sudo systemctl enable apache2.  
sudo systemctl enable sshd
```

4. Preventive Action – Configure Apache to auto-restart on runtime failure.

TOPS TECH MODUAL-1 Assessment

Use systemd's service override to add automatic restart on failure:

Method 1 – Create systemd Override File:

```
sudo systemctl edit apache2
```

Add the following in the editor:

```
[Service]
Restart=on-failure
RestartSec=5s
StartLimitIntervalSec=60
StartLimitBurst=3
```

Save and reload:

```
sudo systemctl daemon-reload
sudo systemctl restart apache2
```

Method 2 – Verify with systemctl show::

```
sudo systemctl show apache2 | grep -i restart
```

Method 3 – Full drop-in config at /etc/systemd/system/apache2.service.d/override.conf::

```
[Service]
Restart=always
RestartSec=10
```

What these settings mean:

- Restart=on-failure: Restarts only if Apache crashes (not manual stop)
- RestartSec=5s: Wait 5 seconds before attempting a restart
- StartLimitBurst=3: Try max 3 restarts within StartLimitIntervalSec

This ensures that if Apache crashes due to a runtime error, it automatically recovers without admin intervention.

TOPS TECH MODUAL-1 Assessment

5. Security – Justify SSH Key-based Authentication over passwords during this vulnerable state.

During a production incident (like this one), the server is especially vulnerable to brute-force attacks because:

- Attacker bots constantly scan for SSH on port 22
- A server that is slow or under load may log attacks late
- Admins may use weak passwords under pressure

Why Key-based Auth is Superior:

1. Brute-Force Resistance:

→ A 4096-bit RSA key has 2^{4096} possible combinations. A brute-force attack is mathematically impossible in any reasonable time.

2. No Password Transmission:

→ Passwords are never sent over the network — even in encrypted form. The key challenge-response never exposes secrets.

3. Two-Factor Security:

→ The private key must be on the developer's machine AND protected by a passphrase. Two things must be compromised, not just one.

4. Eliminates Dictionary Attacks:

→ Dictionary attacks exploit common passwords like 'admin123'. Keys are random dictionaries are useless.

5. Audit Trail:

→ Each developer has a unique key. You can revoke one developer's access without affecting others, and logs clearly show which key was used.

6. Password authentication no:

TOPS TECH MODUAL-1 Assessment

→ Once passwords are disabled in `sshd_config`, even if an attacker guesses the username, they cannot log in without the private key file.

During This Incident Specifically:

- The server is already stressed — password brute-forcing adds extra login attempts to CPU load
- Admins need fast, reliable access — key auth is faster than typing passwords
- If passwords were allowed, attackers could exploit the distracted admin team during the outage.